

agent: discovery-technical-debt-agent cli: Claude Code CLI llm: claude-opus-4-6
run_id: 20260814T114605_gxa4b6 generated_at: 2026-08-14T06:50:51.402Z

8. Technical Debt Analysis

Objective: Establish the prerequisites for an Agentic Harness & Marketplace initiative — code repository health, third-party tool usage, AI tool usage, database usage, and development environment readiness.

Date: 2026-08-14 12:21:01 IST | **Scope:** shende-shweta/FSDKC — PHP 8.3 / Laravel 12 backend, React 19.2 / TypeScript / Vite frontend, Node.js Express dev-API, MariaDB 11 + MongoDB document store, Docker Compose infrastructure, GitHub Actions CI

Executive Summary

EXECUTIVE SUMMARY

The FSDKC repository is a monolithic telecom-QA platform with two product modules (Discovery and Connect) running on a PHP/Laravel backend and React/TypeScript frontend, with a Node.js Express dev-API for local development. The most severe debt is in **repository hygiene**: `composer.lock` is not committed (non-reproducible PHP installs), PHPStan is declared as a dev dependency but never runs in CI or locally, and no linter, formatter, or pre-commit hook is configured anywhere in the project — code style enforcement is completely absent. The **development environment** is partially containerized via Docker Compose but the backend `.env.example` hardcodes `secret` as the database password and there is no root-level `.env.example` to unify the three sub-projects. **Database schema is managed via a single raw SQL init file** with no migration framework, no rollback path, and no per-domain ownership boundaries. On the positive side, CI does exist (GitHub Actions runs PHPUnit and frontend build), Docker Compose orchestration is functional, and the AGENTS.md files show early investment in AI-assisted development conventions. The codebase is **not yet ready** for agentic-harness adoption: the missing lock file, absent style enforcement, and manual schema management must be resolved first.

Yes

TOP-LEVEL .GITIGNORE
PRESENT

1

CI/CD WORKFLOWS FOUND

10 / 10

THIRD-PARTY PACKAGES
DECLARED / WIRED

Partial

.ENV.EXAMPLE PRESENT

OVERALL CODEBASE RATING — TECHNICAL DEBT & AGENTIC READINESS

High Risk

Driven by D1 (Code Repository Health) — missing composer.lock, no code style enforcement, PHPStan declared but never executed — and D4 (Database Usage) — single raw SQL init file with no migration framework or rollback path.

Readiness Benchmark Ratings

#	Dimension	GOOD	MODERATE	HIGH RISK	Measured	Rating
D1	Code Repository Health	all checks pass	1–2 gaps	3+ gaps / no CI	<code>composer.lock</code> missing; PHPStan declared but never run; no linter/formatter/pre-commit hook; no CODEOWNERS or PR template (4 gaps)	HIGH RISK
D2	Third-Party Tool Usage	mostly wired & current	some unused/unwired	many unused/unmaintained	All 10 declared packages are imported and used in application code; PHPStan is a dev dependency but never invoked (1 unwired)	MODERATE
D3	AI Tool / Agentic Readiness	ready	partial	not ready	AGENTS.md present with conventions; <code>.kiro/</code> directory scaffolded; two product modules are structurally uniform — but no CI lint gate, no CLAUDE.md, no automated scaffolding scripts	MODERATE
D4	Database Usage	sound	some gaps	no constraints / shared flat schema	Single raw SQL init file for all tables; no migration framework; no rollback guards; no per-domain schema ownership; MongoDB collections have no schema validation	HIGH RISK
D5	Development Environment	reproducible	partial	manual / fragile	Docker Compose works; <code>.env.example</code> present for backend and dev-api but not at root; no root <code>.env.example</code> unifying the three sub-projects; hardcoded <code>secret</code> password in <code>.env.example</code> and <code>docker-compose.yml</code> ; no linter/formatter enforced	MODERATE

No additional readiness gaps beyond the standard dimensions were observed.

8.1 Code Repository

Check	Finding	Files Inspected	Consequence
-------	---------	-----------------	-------------

<code>.gitignore</code> coverage	Present — covers <code>backend/vendor/</code> , <code>node_modules/</code> , <code>frontend/dist/</code> , <code>.env</code> files, <code>dev-api/.env</code> , and <code>*.log</code> (8 entries).	<code>.gitignore:1-8</code>	Adequate for current stack; prevents committing dependency dirs and env secrets.
CI/CD presence	Present — single workflow <code>ci.yml</code> with two jobs. Backend job: PHP 8.3 setup, MongoDB extension install, <code>composer install</code> , PHPUnit. Frontend job: Node 22, <code>npm ci</code> , <code>tsc --noEmit && vite build</code> . Triggers on push to <code>main</code> / <code>master</code> and all PRs. No lint step, no PHPStan step, and no dev-api tests.	<code>.github/workflows/ci.yml:1-55</code>	CI catches test failures and build breaks but cannot verify code style, static types, or coverage thresholds.
Branch protection signals	Missing — no <code>CODEOWNERS</code> file, no PR template, no required-checks config visible in the repo. Only file under <code>.github/</code> is <code>workflows/ci.yml</code> .	<code>.github/</code> directory	Unreviewed code can land on main; no code-ownership assignment for review routing.
Lock files committed	Partial — <code>package-lock.json</code> committed for root, frontend, and dev-api (3 JS lock files). <code>composer.lock</code> is not committed for the PHP backend. <code>.gitignore</code> does not list <code>composer.lock</code> — it was simply never generated/committed.	<code>package-lock.json</code> (root), <code>frontend/package-lock.json</code> , <code>dev-api/package-lock.json</code> — present; <code>backend/composer.lock</code> — absent	PHP dependency installs are non-reproducible: <code>composer install</code> resolves versions dynamically, risking different packages on CI vs. local vs. production.
Code style enforcement	Completely absent — no ESLint, Prettier, PHP-CS-Fixer, Laravel Pint, EditorConfig, or any linter/formatter config found anywhere. PHPStan is declared in <code>backend/composer.json:12</code> (<code>"phpstan/phpstan": "^2.0"</code>) but has no <code>phpstan.neon</code> config and is never invoked in CI or scripts. No pre-commit hooks (no <code>.husky/</code> , <code>lint-staged</code> , or <code>.pre-commit-config*</code>).	Searched entire repo for <code>.eslintrc*</code> , <code>eslint.config*</code> , <code>.prettier*</code> , <code>.editorconfig</code> , <code>phpcs.xml</code> , <code>pint.json</code> , <code>.php-cs-fixer*</code> , <code>phpstan.neon*</code> , <code>.husky/</code> , <code>lint-staged*</code> , <code>.pre-commit-config*</code> — all absent	Code style is entirely trust-based; formatting inconsistencies will accumulate across contributors. PHPStan's static-analysis value is wasted since it is never executed.

8.2 Third-Party Tools Usage

Backend (PHP — `backend/composer.json`)

Package	Declared	Actually Wired?	Debt
<code>laravel/framework</code> ^12.0	Y	Y — all controllers, models, routing, config, service container use Laravel (13 files import <code>Illuminate*</code> or <code>App*</code> namespaces)	None

<code>mongodb/mongodb</code> ^2.0	Y	Y — <code>app/Services/MongoService.php</code> creates <code>MongoDB\Client</code> , manages 3 collections (<code>transcripts</code> , <code>test_events</code> , <code>call_diagnostics</code>); used by <code>MongoController</code> and <code>RealTimeTestService</code>	None
<code>phpunit/phpunit</code> ^11.0 (dev)	Y	Y — <code>phpunit.xml</code> configured, 2 test files exist (<code>HealthTest.php</code> , <code>ReachabilityCalculationTest.php</code>), CI runs <code>vendor/bin/phpunit</code>	Tests are trivial — one asserts a hardcoded array, the other uses <code>random_int()</code> making it non-deterministic
<code>phpstan/phpstan</code> ^2.0 (dev)	Y	N — no <code>phpstan.neon</code> config file, not referenced in any CI step, not in any composer script	Declared but completely unused; occupying dependency space without providing value

Frontend (`frontend/package.json`)

Package	Declared	Actually Wired?	Debt
<code>react</code> ^19.2.3	Y	Y — <code>main.tsx</code> , all pages and components import React	None
<code>react-dom</code> ^19.2.3	Y	Y — <code>main.tsx</code> uses <code>ReactDOM.createRoot</code>	None
<code>react-router-dom</code> ^7.1.0	Y	Y — <code>App.tsx</code> defines routes for Dashboard, Discovery, Connect pages	Routes defined inline in <code>App.tsx</code> , no centralized <code>routes.ts</code>
<code>@tanstack/react-query</code> ^5.62.0	Y	Y — <code>useQuery</code> / <code>useMutation</code> used in page components for data fetching	None
<code>zustand</code> ^5.0.2	Y	Y — <code>src/store/uiStore.ts</code> implements UI state store	None

Dev-API (`dev-api/package.json`)

Package	Declared	Actually Wired?	Debt
<code>express</code> ^4.21.2	Y	Y — <code>src/server.js</code> creates Express app with full routing	None
<code>cors</code> ^2.8.5	Y	Y — <code>src/server.js:17</code> <code>app.use(cors())</code> with default config	Configured with wildcard <code>*</code> origin — allows any origin to call the API
<code>dotenv</code> ^16.4.7	Y	Y — <code>src/seed.js:1</code> <code>import 'dotenv/config'</code>	Also using <code>node --env-file=.env</code> flag in npm scripts (redundant loading)
<code>mongodb</code> ^6.12.0	Y	Y — <code>src/mongo.js</code> creates <code>MongoClient</code> , manages connections and collections	None
<code>mongodb-memory-server</code> ^10.1.4	Y	Y — <code>src/mongo.js:34</code> falls back to in-memory MongoDB when <code>MONGODB_URI</code> is absent	None — good DX pattern for zero-config local development

Summary: All 10 runtime and dev packages are genuinely imported and used in application code. PHPStan is the sole declared-but-never-invoked dependency — it ships as a dev dependency but provides zero value without a config file or CI integration.

8.3 AI Tool Usage & Agentic Readiness

Check	Finding	Files Inspected
AGENTS.md	Present — root <code>AGENTS.md</code> documents platform stack (Laravel 12, React 19.2, MariaDB + MongoDB), two product modules (Discovery, Connect), and 3 coding conventions. References per-module AGENTS.md files at <code>backend/app/Modules/*/AGENTS.md</code> and <code>frontend/src/modules/*/AGENTS.md</code> .	<code>AGENTS.md:1-24</code>
<code>.kiro/</code> directory	Present — <code>.kiro/settings/mcp-bundles/.gitignore</code> exists, indicating Kiro IDE scaffolding was initialized but no MCP bundles are configured yet.	<code>.kiro/settings/mcp-bundles/.gitignore</code>
CLAUDE.md	Missing — no CLAUDE.md found at any level of the repository.	Searched entire repo
<code>.cursor/</code> directory	Missing — no Cursor IDE configuration present.	Searched entire repo
GitHub Copilot config	Missing — no <code>.github/copilot*</code> files.	<code>.github/</code> directory
Codegen / scaffolding scripts	Missing — no <code>artisan make:*</code> wrappers, no custom generators, no scaffolding automation. Only scripts are <code>npm run dev</code> and <code>npm run install:all</code> .	<code>package.json</code> , <code>backend/composer.json</code> , <code>frontend/package.json</code> , <code>dev-api/package.json</code> — scripts sections
Codebase audit document	Present — <code>docs/CODEBASE_AUDIT_ISSUES.md</code> is a thorough 14-point audit checklist covering backend architecture debt (business logic in controllers, <code>extract()</code> usage, missing service/repository pattern, duplicated code), frontend issues (no centralized API layer, oversized components, missing error boundaries, lifecycle cleanup gaps), security gaps (wildcard CORS, no auth, no validation, no rate limiting), and testing gaps (non-deterministic tests, missing coverage for critical logic, zero frontend tests).	<code>docs/CODEBASE_AUDIT_ISSUES.md:1-198</code>

Convention violations found:

2 files affected.

File	Line(s)	Issue	Action
<code>backend/app/Http/Controllers/Api/LegacyReportController.php</code>	22	<code>extract()</code> usage violates AGENTS.md convention — "No <code>extract()</code> in PHP"	Replace <code>extract()</code> with explicit array destructuring
<code>backend/app/Legacy/LegacyDataMapper.php</code>	6, 12, 24	<code>extract()</code> usage violates AGENTS.md convention — "No <code>extract()</code> in PHP"	Replace <code>extract()</code> with explicit array destructuring

The AGENTS.md explicitly states "No `extract()` in PHP — use explicit destructuring" but `extract()` appears in 2 production files: `backend/app/Legacy/LegacyDataMapper.php:12,24` and `backend/app/Http/Controllers/Api/LegacyReportController.php:21`. The second case is especially dangerous — `extract($request->all())` on raw request input injects all request parameters as local variables.

Agentic readiness assessment: The two product modules (Discovery and Connect) are structurally uniform — each has a controller, model(s), service file, frontend page, and API routes following the same pattern. This uniformity makes them excellent candidates for systematic agent-driven refactoring (e.g. extracting repositories, splitting monolithic page components, deduplicating `buildTree()` logic). However, three prerequisites block safe agent handoff: (1) no CI lint gate means agent-authored code

cannot be automatically style-verified; (2) the missing `composer.lock` means agent-authored dependency changes cannot be verified reproducibly; (3) the existing audit document (`CODEBASE_AUDIT_ISSUES.md`) provides a ready-made work queue but there is no automated harness to process it. The AGENTS.md conventions are a strong starting point but need CI enforcement and a `CLAUDE.md` to be actionable for an automated agent.

8.4 Database Usage

Check	Finding	Files Inspected	Consequence
Schema design (MariaDB)	Adequate — 5 tables with appropriate column types: <code>ENUM</code> constraints for status fields (<code>pending</code> / <code>running</code> / <code>completed</code> / <code>failed</code> on <code>discovery_jobs</code> ; <code>active</code> / <code>paused</code> / <code>alert</code> on <code>connect_monitors</code>), <code>DECIMAL(5,2)</code> for reachability percentages, <code>JSON</code> column for languages, <code>TIMESTAMP</code> for audit fields. Foreign keys present on <code>discovery_nodes.discovery_job_id</code> → <code>discovery_jobs(id)</code> ON DELETE CASCADE and <code>connect_check_results.connect_monitor_id</code> → <code>connect_monitors(id)</code> ON DELETE CASCADE . Missing: no secondary indexes beyond PKs and FKs; no index on <code>discovery_jobs.status</code> or <code>connect_monitors.country_code</code> ; self-referencing FK on <code>discovery_nodes.parent_id</code> is not enforced.	<code>docker/mariadb/init.sql:1-63</code>	Missing secondary indexes will degrade query performance as data grows. The unconstrained <code>parent_id</code> allows orphaned tree nodes.
Schema design (MongoDB)	Basic — 3 collections (<code>transcripts</code> , <code>call_diagnostics</code> , <code>test_events</code>) with compound indexes created in both <code>docker/mongodb/init.js:63-64</code> and <code>dev-api/src/mongo.js:44-46</code> . No JSON Schema validation rules defined on any collection.	<code>docker/mongodb/init.js:1-67</code> , <code>dev-api/src/mongo.js:44-46</code>	Without schema validation, any shape of document can be inserted into any collection, risking silent data corruption if a code change alters the document structure.
Migration hygiene	High Risk — schema is managed via a single <code>docker/mariadb/init.sql</code> file using <code>CREATE TABLE IF NOT EXISTS</code> . No Laravel migrations exist (<code>backend/database/migrations/</code> directory is absent), no versioning, no rollback mechanism. The README explicitly states "Schema is applied via manual SQL init (<code>docker/mariadb/init.sql</code>) — matching Klearcom's manual migration practice." The AGENTS.md convention also codifies this: "Schema via manual SQL, not Laravel migrations."	<code>docker/mariadb/init.sql:1-2</code> , <code>README.md:17</code> , <code>AGENTS.md:24</code>	Any schema change requires editing the single init file and recreating the database container from scratch. There is no way to apply incremental changes, roll back a failed migration, or track which

			schema version is deployed.
Data ownership	Flat — all 5 MariaDB tables live in a single <code>klearcom</code> database with no schema or table-prefix separation between Discovery and Connect modules. MongoDB uses a single <code>klearcom</code> database for both modules, differentiated only by a <code>module</code> string field on each document. The <code>users</code> table exists but has no FK relationship to <code>discovery_jobs</code> or <code>connect_monitors</code> — there is no data ownership chain.	<code>docker/mariadb/init.sql:1-63</code> , <code>docker/mongodb/init.js:1-39</code>	Tables and collections are not scoped per domain, which blocks safe extraction into separate services. Any query can access any module's data without constraint.
Seed/sample data hygiene	Adequate — MariaDB init file includes sample data with <code>INSERT INTO</code> statements using synthetic data (<code>admin@klearcom.local</code> , <code>+18005551234</code> , <code>Bank IVR Discovery</code>). MongoDB dev-api seed (<code>dev-api/src/mongo.js:134-160</code>) is idempotent: checks existing document count, supports <code>--force</code> flag, tags seeded documents with <code>app: 'klearcom'</code> for targeted cleanup. No production data or secrets baked into seeds.	<code>docker/mariadb/init.sql:64-86</code> , <code>dev-api/src/seed-data.js:1-119</code> , <code>dev-api/src/mongo.js:134-160</code>	MariaDB seeds are not independently idempotent — they run only as part of Docker container initialization. Re-seeding requires destroying and recreating the volume.

8.5 Development Environment

Check	Finding	Files Inspected	Consequence
<code>.env.example</code>	Partial — <code>backend/.env.example</code> (14 lines) and <code>dev-api/.env.example</code> (2 lines) both exist. No root-level <code>.env.example</code> exists to document the unified setup. <code>backend/.env.example</code> hardcodes <code>DB_PASSWORD=secret</code> which is the same value used in <code>docker-compose.yml:40</code> — functional but normalizes weak credentials. <code>docker-compose.yml</code> also hardcodes <code>MYSQL_ROOT_PASSWORD: root</code> and <code>MYSQL_PASSWORD: secret</code> directly in the service definition.	<code>backend/.env.example:1-14</code> , <code>dev-api/.env.example:1-2</code> , <code>docker-compose.yml:37-40</code>	New contributors must discover the two sub-project env files independently. Hardcoded passwords in version control train developers to skip credential management.

OS portability	Good — setup instructions support both Docker Compose (any OS with Docker) and direct Node.js execution (<code>npm run dev</code>). No OS-specific tools required. <code>entrypoint.sh</code> uses POSIX-compatible <code>sh</code> .	<code>README.md:19-55</code> , <code>package.json:4-6</code> , <code>docker/php/entrypoint.sh:1-15</code>	No portability blockers.
Containerization	Good — <code>docker-compose.yml</code> defines 5 services (nginx, app, mariadb, mongodb, frontend) with <code>healthcheck</code> on MariaDB (<code>healthcheck.sh --connect --innodb_initialized</code>), named volumes for data persistence (<code>mariadb_data</code> , <code>mongodb_data</code>), and an entrypoint script that handles first-run composer install and <code>.env</code> generation. Frontend Dockerfile uses <code>node:22-alpine</code> . PHP Dockerfile installs <code>pdo_mysql</code> , <code>zip</code> , and <code>mongodb</code> extensions.	<code>docker-compose.yml:1-75</code> , <code>docker/php/Dockerfile:1-17</code> , <code>frontend/Dockerfile:1-7</code> , <code>docker/php/entrypoint.sh:1-15</code>	Functional container setup for local development. The frontend Dockerfile runs the dev server (not a production build) — acceptable for development but not suitable for production deployment as-is.
Code style enforcement	Completely absent — no linter, formatter, EditorConfig, or pre-commit hook is configured. PHPStan is declared but has no config file and is never invoked. No ESLint or Prettier for the frontend/dev-api JavaScript/TypeScript code. The frontend has TypeScript with <code>tsc --noEmit</code> in the build script, but this only catches type errors — not style or quality issues.	Searched entire repo for all common config filenames (<code>.eslintrc*</code> , <code>eslint.config*</code> , <code>.prettier*</code> , <code>.editorconfig</code> , <code>phpcs.xml</code> , <code>pint.json</code> , <code>.php-cs-fixer*</code> , <code>phpstan.neon*</code> , <code>.husky/</code> , <code>lint-staged*</code> , <code>.pre-commit-config*</code>) — all absent	Code style is entirely trust-based; formatting inconsistencies will accumulate across contributors. Mixed JSX/TSX file extensions already exist (<code>LegacyMonitorPoller.jsx</code> alongside <code>.tsx</code> files) with no enforcement.

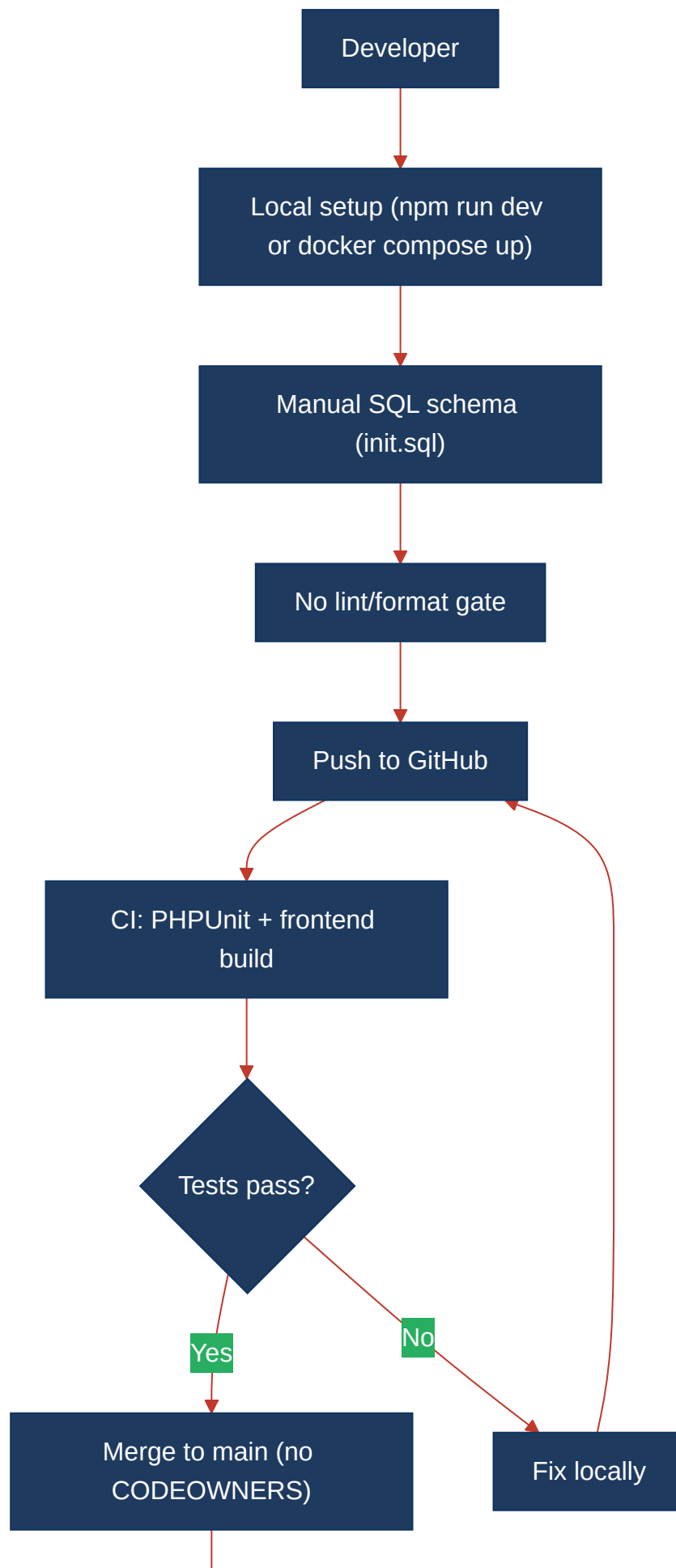
8.6 Prerequisites for Agentic Harness & Marketplace Readiness

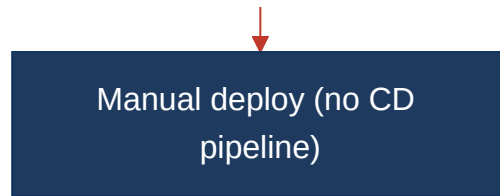
Prerequisite	Current State	Gap
Enumerable work queue (clear, listable units of refactor work)	Two product modules (Discovery, Connect) with symmetric structure (controller → model → page → routes); 14-point audit checklist in <code>docs/CODEBASE_AUDIT_ISSUES.md</code> provides concrete, actionable work items	LOW — audit document already lists concrete work items; module symmetry enables templated refactoring
Isolated, verifiable units of work	Modules share the same database with no ownership boundaries; business logic is duplicated across controllers and the dev-api (<code>buildTree()</code> exists in 3 places, reachability formula in 3 places); page components are monolithic (176–222 lines)	HIGH — no service/repository layer isolation; changes to shared KPI logic or <code>buildTree()</code> affect both modules simultaneously
CI gate to accept agent-authored	GitHub Actions runs PHPUnit and frontend build; no lint step, no PHPStan step, no dev-api tests, no coverage gate, no required-checks	CRITICAL — CI can detect build failures and test regressions but cannot

output	enforcement	verify code style, static analysis, or coverage thresholds for agent-authored changes
Repo hygiene for automation (clean checkout, no secrets)	<code>.gitignore</code> covers deps and env files; <code>.env</code> files are not committed; however <code>composer.lock</code> is missing (non-reproducible installs) and <code>docker-compose.yml</code> contains hardcoded <code>secret</code> / <code>root</code> passwords in plain text	HIGH — non-reproducible PHP installs break deterministic CI; inline credentials in version control
Marketplace packaging readiness	Monorepo with 3 sub-projects (backend, frontend, dev-api) and Docker Compose; no workspace protocol (npm/pnpm workspaces), no shared type definitions, no API contract specifications (OpenAPI/Swagger), no versioning strategy	MEDIUM — functional but not packageable as independent marketplace modules without workspace restructuring and API contracts

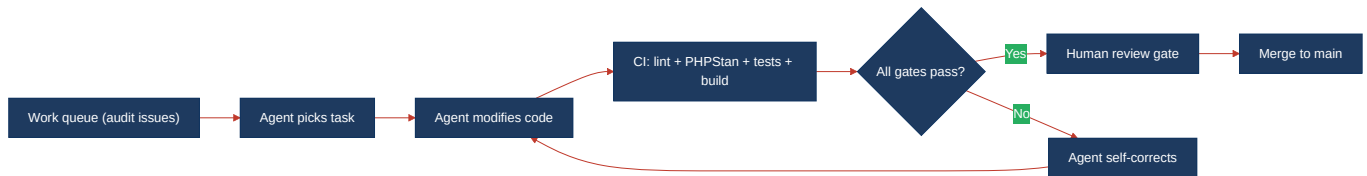
8.7 Diagrams

Current dev / delivery flow

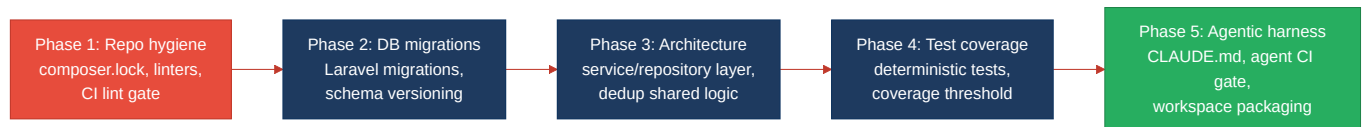




Agentic harness readiness target



Improvement roadmap



8.8 Actions Required

Gap	Action	Rating	Priority
<code>composer.lock</code> not committed	Run <code>composer install</code> in the backend directory and commit the generated <code>composer.lock</code> ; add <code>composer.lock</code> to CI cache key for reproducible installs	HIGH RISK	CRITICAL
No code style enforcement (lint/format)	Add ESLint + Prettier for frontend/dev-api, Laravel Pint or PHP-CS-Fixer for backend; create <code>.editorconfig</code> for baseline indent/whitespace rules; add lint step to <code>.github/workflows/ci.yml</code> before test/build steps	HIGH RISK	CRITICAL
PHPStan declared but never run	Create <code>backend/phpstan.neon</code> with level 5+ baseline; add <code>vendor/bin/phpstan analyse</code> step to CI workflow after PHPUnit; add a composer script <code>"analyse": "phpstan analyse"</code>	HIGH RISK	CRITICAL
No migration framework — single raw SQL init	Introduce Laravel migrations (<code>php artisan make:migration</code>); convert <code>docker/mariadb/init.sql</code> DDL into versioned migration files; add <code>php artisan migrate</code> to CI backend job and Docker <code>entrypoint.sh</code>	HIGH RISK	CRITICAL
No CODEOWNERS or PR template	Add <code>.github/CODEOWNERS</code> mapping <code>backend/</code> and <code>frontend/</code> to respective team leads; add <code>.github/pull_request_template.md</code> with review checklist	HIGH RISK	HIGH
No rollback guards on schema changes	Ensure each Laravel migration has a <code>down()</code> method; add a CI step that runs <code>migrate</code> then <code>migrate:rollback</code> to verify reversibility	HIGH RISK	HIGH
Hardcoded passwords in docker-compose.yml and .env.example	Replace inline <code>MYSQL_PASSWORD: secret</code> and <code>MYSQL_ROOT_PASSWORD: root</code> in <code>docker-compose.yml</code> with	MODERATE	HIGH

	<code>env_file</code> directive pointing to <code>.env</code> ; use placeholder values in <code>.env.example</code> (e.g. <code>DB_PASSWORD=changeme</code>)		
CORS wildcard on both backend and dev-api	Restrict <code>allowed_origins</code> in <code>backend/config/cors.php:6</code> from <code>['*']</code> to specific frontend origins; replace <code>app.use(cors())</code> in <code>dev-api/src/server.js:17</code> with origin-specific config	MODERATE	HIGH
Flat data ownership (no per-domain schema boundaries)	Document table ownership per module (<code>discovery_jobs</code> + <code>discovery_nodes</code> → Discovery; <code>connect_monitors</code> + <code>connect_check_results</code> → Connect); consider separate MongoDB databases per module; add FK from <code>discovery_nodes.parent_id</code> to <code>discovery_nodes.id</code>	HIGH RISK	MEDIUM
MongoDB collections lack schema validation	Add JSON Schema validators to <code>transcripts</code> , <code>test_events</code> , and <code>call_diagnostics</code> collections via <code>db.createCollection()</code> with <code>validator</code> option in <code>docker/mongodb/init.js</code>	HIGH RISK	MEDIUM
No root-level <code>.env.example</code>	Create a root <code>.env.example</code> that documents all required variables across the three sub-projects (backend, frontend, dev-api) with placeholder values	MODERATE	MEDIUM
No CLAUDE.md for agentic harness	Create <code>CLAUDE.md</code> at repo root documenting project conventions, build/test commands, and agent guardrails; extend existing <code>AGENTS.md</code> with CI-verifiable rules	MODERATE	MEDIUM
CI does not run dev-api tests	Add a <code>dev-api</code> job to <code>ci.yml</code> that installs dependencies and runs test suite (create basic API smoke tests first since none currently exist)	MODERATE	MEDIUM

8.9 Expected Outcomes

- **Reproducible builds:** Committing `composer.lock` and enforcing lock-file integrity in CI ensures every environment installs identical dependency versions, eliminating "works on my machine" failures.
- **Automated style enforcement:** Configuring linters/formatters with CI gates eliminates code-style drift and enables agents to produce style-compliant output without human review overhead.
- **Versioned, reversible schema changes:** Migrating from raw SQL init to Laravel migrations provides an auditable history of schema changes with rollback capability, a prerequisite for safe agent-driven database modifications.
- **CI trust for agent-authored changes:** Adding lint, PHPStan, and coverage gates to CI gives the harness a reliable pass/fail signal to accept or reject agent output automatically.
- **Foundation for agentic harness adoption:** With reproducible installs, enforced style, versioned schema, and comprehensive CI gates, the codebase becomes safe for automated agents to propose, verify, and land changes under human review.